Causal Mechanistic Interpretability: From Inspection to Event-Historical Structures, Two-Channel Mediation, and Resilience in Neural and Knowledge Systems

Flyxion

December 2025

Abstract

Recent advances in mechanistic interpretability have reframed the study of neural networks around causal reasoning, progressing from descriptive inspection toward interventions, counterfactuals, and algorithmic abstraction. However, most current approaches remain implicitly state-centric, treating internal values as the sole carriers of causal influence while overlooking irreversibility, history sensitivity, and regime change. This essay develops a unified causal framework that distinguishes *texture-nodes*, which carry instantaneous values, from *event-nodes*, which encode regime-selecting and future-constraining commitments. We formalize a two-channel mediation theory separating value-level and event-level causal effects, introduce graph rewrite semantics for regime change, and clarify the structural meaning of substitution failure and algorithmic mixture. Finally, we extend the framework beyond neural networks to large-scale knowledge systems, showing that resilience in such systems depends critically on event-level structure rather than redundancy of content alone. The resulting account reconceives mechanistic understanding as event-historical causal reverse engineering, preserving empirical rigor while incorporating time, constraint, and irreversibility as first-class explanatory elements.

Contents

1 Introduction

The rapid deployment of large neural networks across scientific, economic, and cultural domains has intensified the demand for explanations of how these systems work. Early interpretability efforts focused on visualization, attribution, and pattern discovery: neurons were associated with concepts, attention maps were inspected for semantic alignment, and representations were probed for linear structure. While informative, these approaches struggled to distinguish correlation from causation, often leaving unanswered whether an observed internal pattern was functionally responsible for behavior or merely an epiphenomenal trace.

Mechanistic interpretability emerged in response to this limitation, proposing that genuine understanding requires identifying internal mechanisms whose manipulation produces predictable

changes in behavior. This shift replaced inspection with intervention, emphasizing counterfactual reasoning and causal testing. The goal became not simply to describe what activations look like, but to reverse-engineer the algorithms implemented by neural networks.

A coherent causal paradigm has since formed, drawing heavily on ideas from classical causality and mediation analysis. Researchers now routinely intervene on internal variables, restore corrupted components, and test algorithmic hypotheses via interchange interventions. Despite this progress, important conceptual tensions remain. Most notably, existing frameworks implicitly assume that causal influence is exhausted by instantaneous state variables. History, irreversibility, and regime change are treated as secondary or external concerns.

This essay argues that these omissions are not merely philosophical but technically limiting. We show that many empirical phenomena already observed in mechanistic interpretabilitysuch as substitution failure, block-level mediation, heuristic mixture, and narrow domain validityare difficult to formalize without distinguishing between value-level mediation and event-level constraint. By introducing a minimal typed extension to causal graphs, we reconcile these phenomena within a unified framework that remains compatible with existing methods.

2 What Does "Mechanistic" Mean?

The term *mechanistic* is used heterogeneously in the interpretability literature. In some contexts, it functions as a cultural marker, referring to work motivated by AI safety or philosophical concerns about alignment and risk. In others, it is used broadly to describe any analysis that looks inside a model rather than treating it as a black box. Neither usage provides a principled criterion for understanding.

A more precise technical definition ties mechanism to causality. Under this view, a system is mechanistically understood if its behavior can be explained in terms of internal components whose causal roles are demonstrable through intervention. Observation alone is insufficient: a component that correlates with behavior but cannot be shown to affect it under intervention does not constitute a mechanism.

This definition aligns mechanistic interpretability with the scientific practice of explanation rather than with visualization or intuition. It also sets a demanding standard: explanations must support counterfactual reasoning and survive substitution tests. The remainder of this essay adopts this narrow technical sense of mechanism.

3 Causality as the Foundation of Mechanism

Causality provides the formal language needed to move from correlation to explanation. The defining feature of causal reasoning is the intervention: an operation that sets a variable to a value independently of its usual causes.

Definition 1 (Intervention)**.** *An intervention on a variable X sets $X := x$ while severing its dependence on parent variables, denoted* $\mathrm{do}(X := x)$.

In neural networks, interventions may target inputs, internal activations, attention heads, or entire modules. A component is causally relevant if intervening on it produces systematic changes in output behavior.

Causal reasoning also supports counterfactuals: questions about what would have happened had some internal variable taken a different value. Counterfactuals are essential for testing necessity, sufficiency, and equivalence between mechanisms.

This causal foundation underlies all subsequent developments, from activation steering to causal abstraction.

4 Activation Steering as Linear Intervention

Activation steering represents one of the earliest intervention-based techniques in interpretability. By computing average activation differences between two classes of behavior and adding the resulting vector to internal representations, researchers can bias model outputs toward one behavior or another.

Steering demonstrates that internal representations are not inert: small, targeted changes can systematically alter behavior. In causal terms, this establishes that certain directions in activation space are causally efficacious.

However, steering remains limited. It does not identify the algorithm being implemented, nor does it explain why a particular direction works. Moreover, steering effects are often brittle and domain-specific, and they are typically less effective than prompting or fine-tuning for control. Steering thus serves primarily as evidence that representations matter, not as a full mechanistic explanation.

5 Causal Mediation and Tracing Information Flow

Causal mediation analysis predates mechanistic interpretability and originates in fields such as medicine and social science. Given an input X, an output Y, and an intermediate variable Z, mediation analysis asks whether the effect of X on Y flows through Z.

In neural networks, this framework has been adapted through causal tracing. A typical causal tracing experiment corrupts the input (e.g. by adding noise to token embeddings), observes degraded performance, and then selectively restores internal components to their clean values. Components whose restoration recovers performance are identified as mediators.

These experiments reveal structured pathways: early layers retrieve information, intermediate layers transform it, and later layers route it to outputs. Importantly, these claims are grounded in intervention rather than correlation.

Nevertheless, causal tracing typically measures only whether restoring a value recovers output behavior. It does not directly test whether the same *computational regime* has been restored, a distinction that becomes crucial in more complex settings.

6 Causal Abstraction and Algorithmic Understanding

Causal abstraction seeks to relate neural networks to human-understandable algorithms. The central idea is to treat both the network and the algorithm as causal models and to test whether interventions at corresponding points produce equivalent effects.

Definition 2 (Causal Abstraction)**.** *A high-level model $\mathcal{A}$ is a causal abstraction of a system $\mathcal{N}$ if there exists a mapping between their variables such that interventions on $\mathcal{A}$ correspond to interventions on $\mathcal{N}$ with equivalent outcomes.*

Interchange interventions are the primary tool for testing abstraction. By replacing internal values with those drawn from different inputs, researchers can test whether the system behaves as the abstract algorithm predicts. Success indicates interventional equivalence, not merely functional similarity.

Causal abstraction provides the strongest form of mechanistic claim currently available. Yet even here, abstraction is evaluated primarily at the level of values, leaving open questions about history, irreversibility, and regime change.

7 Limits of State-Based Approaches

State-based causal models treat systems as fully characterized by their current variables. History matters only insofar as it influences present values. This assumption is often reasonable, but it fails in systems where irreversible commitments alter what future states are possible.

In neural networks, identical activation states may arise through different internal processes. While these states may behave identically under some interventions, they may diverge under others because earlier events have constrained the system differently. State-based models lack the vocabulary to express this distinction.

More broadly, state-centric functionalism collapses history into function, ignoring thermodynamic, informational, and organizational constraints. To capture these phenomena, causal explanation must be extended to include events that alter the space of possible futures.

8 Event-Nodes and Texture-Nodes

We introduce a minimal typed extension to causal graphs.

Definition 3 (Texture-Node)**.** *A texture-node is a causal variable whose influence is fully determined by the instantaneous value it carries, $X_v \in \mathsf{Val}_v$.*

Definition 4 (Event-Node)**.** *An event-node is a causal variable whose value selects a regime $E_v \in \mathsf{Reg}_v$, constraining the set of admissible future states* $\mathrm{Adm}(\mathcal{H}_t)$.

Texture-nodes correspond to activations, vectors, and parameters. Event-nodes correspond to regime selection, heuristic choice, gating, or irreversible commitments. The distinction is not metaphysical but operational: texture interventions substitute values, while event interventions alter what substitutions remain meaningful.

9 Typed Causal Graphs and Rewrite Semantics

A typed causal model consists of a graph $\mathcal{G} = (V, E)$ with a typing function $\tau : V \to \{\mathbf{T}, \mathbf{E}\}$. Structural equations update texture-nodes and event-nodes differently.

Event-nodes induce graph rewrite semantics. Selecting a regime rewrites parts of the texture graph, analogous to choosing a generator in procedural graphics. Parameter ratios function as vectors within a generator, but the generator itself determines the space of possible outputs.

This interpretation explains why some substitutions fail: the required generator no longer exists.

10 Two-Channel Causal Mediation

Causal mediation decomposes total effects into distinct channels.

Theorem 1 (Two-Channel Mediation)**.** *Let X be an input, T a texture state, E an event state, and Y an output. Then the total effect of X on Y decomposes as*

$$\mathrm{TE}_Y = \mathrm{ME}_Y^T + \mathrm{ME}_Y^E,$$

where ME_Y^T is the texture-mediated effect and ME_Y^E is the event-mediated effect. Texture mediation suffices if and only if the intervention on X leaves E invariant.

This theorem formalizes when standard causal tracing is complete and when it is not.

11 Irreversibility, History, and Responsibility

Event-nodes introduce structural irreversibility: once a regime is selected, no sequence of texture substitutions can restore excluded futures. History matters as constraint, not narrative.

Responsibility thus includes both enabling and preventing actions. The absence of a pathway can be as causally significant as its presence.

12 Extension to Knowledge Networks and Resilience

The framework extends naturally to knowledge systems such as collaborative encyclopedias. Articles, links, and edits are texture-level structures. Policies, governance decisions, and user removals are event-level interventions.

A knowledge network may retain content while losing resilience if event-level structures constrain future growth or repair. Resilience analysis must therefore track both redundancy and regime stability.

13 Against State-Based Functionalism

State-based functionalism treats systems as equivalent if they implement the same input-output mapping. This erases history and irreversibility. The present framework rejects this collapse, grounding explanation in causal structure over time.

14 Event-Nodes as Spherepop Operators

We now complete the integration by identifying event-nodes with the primitive irreversible operators of the Spherepop calculus. This move does not add new metaphysical assumptions; rather, it refines the causal semantics already implicit in mechanistic interpretability. Spherepop provides a vocabulary for commitments that restrict the space of admissible futures, precisely the role event-nodes were introduced to play.

14.1 Option-Spaces and Kernel Histories

Let $\Omega(\ell)$ denote the *option-space* at layer ℓ: the set of admissible future computational trajectories consistent with all prior events. Unlike a state-space, which enumerates possible configurations at a single time, an option-space encodes structured constraints on how computation may proceed.

Define the *kernel history* at layer ℓ as the ordered sequence of event-nodes:

$$\mathcal{H}_\ell = (e_1, e_2, \ldots, e_\ell),$$

where each e_i is an event-node firing at layer i. The kernel history induces a restriction

$$\Omega(\ell) = \Omega(0) \mid \mathcal{H}_\ell,$$

where $\Omega(0)$ is the unconstrained option-space of syntactically possible computations.

Texture values at layer ℓ must lie in the admissible texture set

$$\mathcal{T}_\ell^{\mathcal{H}} = \{t \in \mathsf{Val}_\ell \mid t \text{ is compatible with } \Omega(\ell)\}.$$

This distinction formalizes a key empirical observation: restoring an activation that is numerically "correct" may still fail if it is incompatible with the kernel history induced by earlier events.

14.2 Pop Events: Irreversible Exclusion

A *Pop* event permanently removes computational pathways.

Definition 5 (Pop Event)**.** *A pop event $e_\ell^{\text{pop}}(p)$ at layer ℓ removes pathway p from the option-space:*

$$\Omega(\ell + 1) = \Omega(\ell) \setminus \{p\}.$$

Pop events are categorical, not graded. They impose infinite cost on the excluded pathway rather than merely down-weighting it.

Neural instantiations. Examples include:

- Causal attention masks enforcing autoregressive order.
- Hard attention masking between tokens.

- Structural pruning of neurons, heads, or edges.
- Dropout interpreted as stochastic pop during training.

Lemma 1 (Pop Irreversibility)**.** *If $e_\ell^{\text{pop}}(p)$ occurs at layer ℓ, then for all texture interventions $\text{do}(T_{\ell'} := t)$ with $\ell' > \ell$, the pathway p remains excluded:*

$$p \notin \Omega(\ell') \quad \forall \ell' > \ell.$$

Proof. Pop events rewrite the computational graph by removing edges or dependencies. Texture interventions operate only on values within the existing graph. Since p is no longer represented structurally, no downstream value substitution can reintroduce it. □

This explains why some causal tracing restorations fail catastrophically: the required pathway has been popped earlier in the kernel history.

14.3 Bind Events: Dependency Formation

A *Bind* event creates structured dependencies or ordering constraints without eliminating options outright.

Definition 6 (Bind Event)**.** *A bind event $e_\ell^{\text{bind}}(x \prec y)$ introduces a precedence or dependency relation that must be respected by all futures in $\Omega(\ell + 1)$.*

Neural instantiations.

- Residual connections enforcing temporal precedence.
- Induction heads binding earlier tokens to later completions.
- Syntax-sensitive attention establishing grammatical relations.

Bind events restrict *how* options may occur rather than *whether* they occur. They increase structure without reducing the cardinality of Ω.

14.4 Refuse Events: Principled Non-Activation

A *Refuse* event encodes deliberate non-action: the exclusion of certain responses despite their representational availability.

Definition 7 (Refuse Event)**.** *A refuse event excludes a subset $A \subseteq \Omega(\ell)$ based on constraint rather than incapacity:*

$$\Omega(\ell + 1) = \Omega(\ell) \setminus A,$$

where A remains representable but inadmissible.

Neural instantiations.

- Safety filters and alignment constraints.
- Logit suppression mechanisms.
- Rule-based overrides embedded in learned policies.

Refusal differs from pop in that the excluded actions remain intelligible to the system. They are excluded by commitment, not ignorance.

14.5 Collapse Events: Quotienting Distinctions

A *Collapse* event identifies distinctions that no longer affect future computation.

Definition 8 (Collapse Event)**.** *A collapse event induces an equivalence relation $\sim$ on $\Omega(\ell)$ and replaces it with the quotient:*

$$\Omega(\ell + 1) = \Omega(\ell)/\sim.$$

Neural instantiations.

- Max and average pooling.
- Dimensionality reduction layers.
- Learned projections with nontrivial kernels.
- Attention as weighted collapse over token sets.

Proposition 1 (Collapse Reduces Optionality)**.** *Let $q : \Omega \to \Omega/\sim$ be a collapse map. Then*

$$|\Omega(\ell + 1)| < |\Omega(\ell)|,$$

and the reduction is irreversible under texture interventions.

Proof. Distinct elements identified under $\sim$ are mapped to a single equivalence class. Since the kernel of q has been quotiented out, no downstream value assignment can reintroduce the lost distinctions. □

Collapse formalizes abstraction and compression as causal operations, not merely representational conveniences.

14.6 Event-Nodes and Two-Channel Mediation

With Spherepop operators in hand, the two-channel mediation theorem admits a precise interpretation. Texture-mediated effects correspond to value propagation within a fixed kernel history $\mathcal{H}$, while event-mediated effects correspond to modifications of $\mathcal{H}$ itself.

Activation patching succeeds when the substituted texture lies in

$$\mathcal{T}_\ell^{\mathcal{H}} \cap \mathcal{T}_\ell^{\mathcal{H}'},$$

and fails otherwise. This explains substitution failure without appealing to ad hoc notions of entanglement or distributedness.

14.7 Mechanistic Interpretability as a Spherepop Process

Finally, the interpretability enterprise itself obeys Spherepop logic. Each experimenteach ablation, patch, or interchange interventionis an irreversible pop in the researcher's hypothesis space. Explanations are not accumulated additively but forged through exclusion.

Mechanistic understanding, on this view, is not the enumeration of states but the reconstruction of the event-history that constrains what the system can do. Neural networks execute Spherepop programs; interpretability reconstructs them.

Remark 1. This reframing resolves a persistent tension in interpretability: why explanation feels historical rather than snapshot-based. Understanding is achieved not by observing what is present, but by inferring what has been irrevocably ruled out.

15 Causal Tracing as Event-History Archaeology

Causal tracing has emerged as one of the most powerful tools in mechanistic interpretability. By corrupting inputs and selectively restoring internal components, researchers identify which parts of a network are sufficient to recover correct behavior. While traditionally interpreted as tracing *information flow*, this section argues that causal tracing is more precisely understood as a form of *event-history archaeology*: an attempt to reconstruct which irreversible commitments were necessary for the observed output.

This reinterpretation explains both the successes and the systematic failure modes of causal tracing, while motivating a principled extension to event-level interventions.

15.1 Standard Causal Tracing Revisited

In its canonical form, causal tracing proceeds as follows:

1. A clean input x produces correct output y.
2. The input is corrupted, $x \to x_{\text{noisy}}$, yielding degraded output.
3. Internal components are restored to their clean values, $\text{do}(T_\ell := t_\ell^{\text{clean}})$.
4. The degree of output recovery is measured.

Components whose restoration substantially recovers y are interpreted as mediators of the causal effect from x to y.

Within the framework developed here, this procedure probes whether the restored texture t_ℓ^{clean} lies within the admissible texture set

$$\mathcal{T}_\ell^{\mathcal{H}'},$$

where $\mathcal{H}'$ is the kernel history induced by the corrupted input.

15.2 Kernel Divergence Under Corruption

Corrupting the input does not merely perturb early activations; it may induce a different sequence of event-nodes. Let $\mathcal{H}$ denote the kernel history induced by the clean input and $\mathcal{H}'$ the history induced by the corrupted input.

In general,

$$\mathcal{H} \neq \mathcal{H}'.$$

This divergence explains a central empirical phenomenon: restoring a value that was sufficient in the clean run may be insufficient or even nonsensical under the corrupted kernel.

Definition 9 (Kernel Compatibility)**.** *A texture value t_ℓ is* kernel-compatible *with history $\mathcal{H}$ if $t_\ell \in \mathcal{T}_\ell^{\mathcal{H}}$.*

Standard causal tracing implicitly assumes kernel compatibility. When this assumption fails, restoration yields partial recovery or complete failure.

15.3 Event-Level Necessity

To formalize this insight, we introduce event-level causal tracing.

Definition 10 (Event Necessity)**.** *An event e_ℓ is necessary for output y if*

$$P(Y = y \mid \text{do}(\mathcal{H} \textit{ includes } e_\ell)) \gg P(Y = y \mid \mathcal{H} \textit{ excludes } e_\ell).$$

Event necessity tests whether a specific irreversible commitment must occur for correct behavior, independent of subsequent texture values.

15.4 Event-Level Causal Tracing Protocol

An event-aware tracing procedure proceeds in two stages:

Stage 1: Kernel Reconstruction

1. Corrupt the input, inducing kernel $\mathcal{H}'$.
2. For each candidate event e_ℓ in the clean history $\mathcal{H}$, force its occurrence:

$$\text{do}(\mathcal{H}' := \mathcal{H}' \cup \{e_\ell\}).$$

3. Measure output recovery.

Stage 2: Texture Restoration Only after restoring the necessary event structure do we restore textures:

$$\mathrm{do}(T_\ell := t_\ell^{\mathrm{clean}}).$$

Proposition 2 (Two-Stage Restoration)**.** *Texture restoration recovers behavior if and only if either:*

1. *the kernel histories agree, $\mathcal{H} = \mathcal{H}'$, or*
2. *event restoration precedes texture restoration.*

Proof. If $\mathcal{H} = \mathcal{H}'$, kernel compatibility holds by definition. If $\mathcal{H} \neq \mathcal{H}'$, texture restoration alone may target values outside $\mathcal{T}_\ell^{\mathcal{H}'}$. Restoring the necessary events aligns the option-space, rendering the texture admissible. □

15.5 Explaining Partial Recovery

Causal tracing experiments often show graded recovery rather than binary success or failure. This follows naturally from partial kernel overlap.

Define the kernel intersection:

$$\Omega_\cap = \Omega(\mathcal{H}) \cap \Omega(\mathcal{H}').$$

Textures lying in $\Omega_\cap$ yield partial recovery; those outside yield none. This predicts block-structured recovery patterns observed empirically, where early-layer restorations sometimes succeed and later-layer restorations fail, or vice versa.

15.6 Relation to Two-Channel Mediation

Causal tracing is thus a probe of both mediation channels:

- **Texture channel**: Does restoring this value help?
- **Event channel**: Is the value even admissible?

Standard tracing collapses these questions. Event-aware tracing separates them.

Remark 2. This reinterpretation does not invalidate existing causal tracing results. Rather, it explains why they work when they do, why they fail when they fail, and how to systematically extend them.

15.7 Archaeology Rather Than Flow

Finally, the metaphor of "information flow" becomes misleading. Causal tracing does not observe information moving through a static pipeline; it infers which irreversible commitments must have occurred for the observed behavior to be possible at all.

Mechanistic interpretability, under this view, resembles archaeology more than microscopy. We reconstruct a past sequence of commitments from present structure, ruling out histories incompatible with the observed effects.

Remark 3. This perspective resolves a persistent puzzle: why mechanistic understanding feels historical rather than instantaneous. What is explained is not a state, but the irreversible path by which the system arrived there.

16 Algorithmic Mixture as Kernel Switching

A recurring empirical observation in mechanistic interpretability is that neural networks appear to implement multiple distinct algorithms for the same task. Small changes in input phrasing, context length, or distributional regime can trigger qualitatively different internal behavior, even when outputs remain superficially similar. Attempts to capture such behavior with a single global causal abstraction often succeed only within narrow domains, failing catastrophically outside their tested regime.

This section argues that such phenomena are best understood not as distributed superpositions of partial algorithms, but as *kernel switching*: event-level selection among multiple distinct event-histories, each corresponding to a different algorithmic regime.

16.1 Empirical Motivation

Evidence for algorithmic mixture arises across tasks and model scales. Examples include:

- Syntax-sensitive tasks where different heuristics dominate depending on surface cues.
- Factual recall that alternates between retrieval-based and pattern-completion strategies.
- In-context learning behaviors that abruptly fail when prompts cross subtle thresholds.

These observations challenge explanations based solely on smooth interpolation in representation space. Instead, they suggest discrete changes in computational structure.

16.2 Kernel Multiplicity

We formalize this by positing a finite or countable set of candidate kernel histories:

$$\{\mathcal{H}^{(1)}, \mathcal{H}^{(2)}, \ldots, \mathcal{H}^{(k)}\},$$

each inducing a distinct option-space $\Omega^{(i)}(\ell)$ and a corresponding family of admissible textures $\mathcal{T}_\ell^{\mathcal{H}^{(i)}}$.

Each kernel $\mathcal{H}^{(i)}$ corresponds to a coherent algorithmic regime: a structured pattern of pop, bind, refuse, and collapse events that constrains future computation in a characteristic way.

Assumption 1 (Kernel Coherence)**.** *Each $\mathcal{H}^{(i)}$ induces a self-consistent option-space such that all admissible textures yield stable behavior under small perturbations.*

This assumption distinguishes kernels from arbitrary mixtures of events; a kernel is not merely a prefix of events, but a viable computational regime.

16.3 Kernel Selection as an Event

Kernel choice is itself an event-level operation.

Definition 11 (Kernel Selection Event)**.** *A kernel selection event is an event-node*

$$e_{select} : X \mapsto \mathcal{H}^{(i)},$$

which commits the computation to kernel $\mathcal{H}^{(i)}$ based on early input features.

Once e_{select} fires, all downstream computation is constrained by the chosen kernel. Texture variation occurs only within $\mathcal{T}_\ell^{\mathcal{H}^{(i)}}$.

Neural instantiations. Kernel selection may be implemented by:

- Early attention heads acting as gatekeepers.
- Routing decisions in mixture-of-experts architectures.
- Implicit gating via saturation or thresholding effects.

These mechanisms need not be explicit; kernel selection may be an emergent consequence of training dynamics.

16.4 Mid-Computation Switching and Hybrid Histories

In some cases, networks appear to switch strategies mid-computation.

Definition 12 (Heuristic Switch Event)**.** *A heuristic switch is an event*

$$e_{switch} : \mathcal{H}^{(i)} \to \mathcal{H}^{(j)}$$

occurring at layer ℓ, yielding a hybrid history

$$\mathcal{H} = \mathcal{H}^{(i)}_{<\ell} \circ e_{switch} \circ \mathcal{H}^{(j)}_{\geq \ell}.$$

Hybrid histories often exhibit instability. Early events constrain the option-space in ways incompatible with later events drawn from a different kernel.

Remark 4. Many brittle failure modes in large language models can be interpreted as the execution of incompatible hybrid kernels.

16.5 Relation to Two-Channel Mediation

Algorithmic mixture clarifies the role of event-mediated effects in two-channel mediation. Texture-mediated effects describe variation *within* a kernel, while event-mediated effects govern which kernel is active.

Misinterpreting kernel switching as distributed texture mediation leads to incorrect abstractions. An intervention that changes kernel selection cannot be captured by a value-only causal model.

16.6 Consequences for Causal Abstraction

Causal abstractions are necessarily kernel-relative.

Proposition 3 (Kernel-Relative Abstraction)**.** *A causal abstraction $\mathcal{A}$ is valid for a system $\mathcal{N}$ if and only if it preserves kernel-selection events:*

$$\mathrm{do}_{\mathcal{A}}(e^{(i)}_{select}) \;\leftrightarrow\; \mathrm{do}_{\mathcal{N}}(e^{(i)}_{select}).$$

Proof. If kernel-selection events are not preserved, interventions that appear equivalent at the texture level may induce different option-spaces, violating interventional equivalence. □

This result explains why many abstractions appear correct under limited testing but fail to generalize: they implicitly assume a fixed kernel.

16.7 Interpretive Implications

Algorithmic mixture is not a defect but a consequence of optimization under limited capacity. Maintaining multiple kernels allows networks to trade off robustness, generalization, and efficiency.

From the perspective of mechanistic interpretability, however, mixture demands humility. There may be no single answer to the question "what algorithm does this network implement?" without specifying the kernel under consideration.

Remark 5. The appropriate unit of explanation is not the network as a whole, but the pair $(\mathcal{N}, \mathcal{H}^{(i)})$: a network executing a particular kernel.

17 Collapse as Learned Abstraction

Abstraction is often treated in interpretability as an epistemic act: the researcher summarizes or simplifies a complex mechanism in order to reason about it. In this section we argue for a stronger claim: neural networks themselves perform abstraction as a *causal operation*. Within the Spherepop framework, abstraction corresponds to *collapse*the irreversible identification of distinctions that no longer affect future computation.

Understanding collapse as a causal event clarifies the relationship between compression, dimensionality reduction, and mechanistic explanation, and it resolves persistent asymmetries observed in causal tracing and activation patching.

17.1 Collapse in Spherepop

Recall that a collapse event induces an equivalence relation on the option-space, replacing it with a quotient. Informally, collapse answers the question: *which distinctions can be forgotten without changing what can still be done?*

Definition 13 (Collapse Event)**.** *A collapse event e^{col}_{ℓ} induces an equivalence relation $\sim$ on $\Omega(\ell)$ and produces*

$$\Omega(\ell+1) = \Omega(\ell)/\!\sim,$$

where futures differing only along collapsed distinctions are identified.

Collapse is irreversible: once two possibilities are identified, no subsequent operation can distinguish them.

17.2 Architectural Collapse

Some collapse operations are explicit in neural architectures.

Pooling. Max pooling collapses spatial distinctions under the equivalence relation "shares the same maximum". Average pooling collapses under additive equivalence. In both cases, multiple configurations map to a single downstream representation.

Normalization. Layer normalization collapses scale distinctions. Two activation vectors differing only by a scalar multiple become indistinguishable after normalization.

These operations are not approximations; they are exact quotient maps.

17.3 Learned Collapse via Linear Projection

More subtly, collapse occurs through learned projections.

Consider a linear map

$$W : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}, \quad d_{\text{out}} < d_{\text{in}}.$$

This map defines an equivalence relation:

$$x \sim_W x' \iff Wx = Wx'.$$

The kernel $\ker(W)$ consists of distinctions that are eliminated by the projection.

Proposition 4 (Learned Collapse)**.** *A learned projection W implements a collapse event whose quotient space is isomorphic to* $\text{im}(W)$.

Proof. All vectors differing by an element of $\ker(W)$ are mapped to the same image. Since downstream computation depends only on Wx, distinctions along $\ker(W)$ are irreversibly removed. □

This reframes dimensionality reduction as causal commitment, not mere parameter efficiency.

17.4 Collapse and Optionality

Collapse reduces optionality: the number of distinct futures the system can pursue.

Let $|\Omega(\ell)|$ denote the cardinality of the option-space (or an appropriate measure thereof).

Lemma 2 (Optionality Reduction)**.** *If e_ℓ^{col} is a nontrivial collapse event, then*

$$|\Omega(\ell + 1)| < |\Omega(\ell)|.$$

This reduction explains why early collapse is especially consequential: distinctions eliminated early cannot support downstream branching.

17.5 Asymmetry in Causal Tracing

Collapse provides a principled explanation for an observed asymmetry in causal tracing experiments. Restoring early-layer activations often fails to recover behavior once later layers have collapsed relevant distinctions, whereas restoring later activations may partially succeed if earlier distinctions remain available.

Within the present framework, this follows immediately. If a collapse event has already quotiented out a distinction, no texture restoration can reintroduce it.

Remark 6. This explains why causal tracing appears more effective when restoring components *before* major collapse events than after.

17.6 Collapse vs. Explanation

Collapse also clarifies the nature of abstraction in explanation. Abstraction is justified when collapsed distinctions are causally irrelevant to future behavior.

Definition 14 (Justified Abstraction)**.** *An abstraction is justified if it corresponds to a collapse event whose quotient preserves all downstream causal effects of interest.*

This criterion distinguishes explanatory abstraction from lossy summary. An abstraction that collapses distinctions still required for some counterfactual reasoning is invalid, even if it predicts observed behavior in a narrow regime.

17.7 Relation to Algorithmic Mixture

Collapse interacts with kernel switching in subtle ways. Different kernels may collapse different distinctions. What is irrelevant in one algorithmic regime may be crucial in another.

This explains why abstractions that are valid within one kernel may break under kernel switching: the collapsed distinctions become relevant again, but cannot be recovered.

Remark 7. Collapse is kernel-relative. There is no globally valid abstraction independent of the event-history in which it is applied.

17.8 Implications for Interpretability

Recognizing collapse as a causal operation sharpens the goals of mechanistic interpretability. The task is not merely to identify where information flows, but to identify where distinctions are irreversibly discardedand to justify why.

In this light, mechanistic understanding consists in reconstructing not only how representations are transformed, but which distinctions the system has committed to forgetting.

18 Worked Transformer Case Study: Event-Aware Mechanistic Analysis

We now ground the preceding theoretical framework in a concrete mechanistic analysis of a transformer model. The goal of this section is not to introduce new conceptual machinery, but to demonstrate how event-aware causal reasoning resolves empirical puzzles that remain opaque under texture-only analysis.

We focus on a canonical task studied in mechanistic interpretability: *indirect object identification* (IOI). This task has the advantage of being simple, well-studied, and known to admit circuit-level explanationsyet it also exhibits substitution failures and regime sensitivity that motivate the present framework.

18.1 Task Definition

Consider sentences of the form:

"Alice gave Bob the book."

The task is to identify the indirect object (IO), here "Bob." We consider a pretrained transformer $\mathcal{N}$ evaluated on a dataset of such sentences with controlled syntactic variation.

Input. Tokenized sentence $x = (x_1, \ldots, x_n)$.

Output. A probability distribution over token positions, with success defined as assigning maximal probability to the correct IO token.

Metric. Accuracy or log-probability assigned to the correct IO.

18.2 Clean Execution and Kernel History

For a clean input x, the network induces a kernel history

$$\mathcal{H} = (e_1, e_2, \ldots, e_L),$$

where L is the number of layers.

Empirical analysis (via ablation and probing) suggests the following event-level structure:

Early layers ($\ell = 1$–3).

- e_1^{bind}: Bind verb token ("gave") to following noun phrases.
- e_2^{pop}: Exclude subject position from IO candidates.

Middle layers ($\ell = 4$–6).

- e_4^{collapse}: Pool candidate noun representations into a role-specific subspace.
- e_5^{meld}: Synthesize positional and semantic cues via multi-head attention.

Late layers ($\ell = 7+$).

- Texture refinement and logit sharpening within the fixed kernel.

This history induces an option-space $\Omega(\ell)$ in which only certain token-role assignments remain admissible.

18.3 Input Corruption and Kernel Divergence

We now corrupt the input by adding noise to the embedding of the verb token:

$$x_{\text{noisy}} = x + \mathcal{N},$$

where $\mathcal{N}$ is Gaussian noise applied to the "gave" token.

Empirically, this corruption often causes a sharp drop in IO accuracy.

Crucially, analysis reveals that the corrupted input induces a *different kernel history* $\mathcal{H}'$, in which:

- The early bind event fails or fires incorrectly.
- The pop event excluding the subject may not occur.
- Subsequent collapse pools over an incorrect candidate set.

Thus $\mathcal{H}' \neq \mathcal{H}$, even when later-layer activations remain close in norm.

18.4 Texture-Only Restoration

We first apply standard causal tracing.

Procedure. For each layer ℓ, we restore the clean activation:

$$\text{do}(T_\ell := t_\ell^{\text{clean}}),$$

while keeping the noisy input.

Observation. Restoration yields:

- Partial recovery when restoring middle-layer activations.
- Little or no recovery when restoring late-layer activations.

- Highly variable results across runs.

Under a texture-only interpretation, these results are puzzling. The restored activations were sufficient in the clean run, yet fail to fully recover behavior here.

18.5 Event-Aware Diagnosis

Within the present framework, the failure is expected.

The restored texture t_ℓ^{clean} is admissible under $\mathcal{H}$ but not under $\mathcal{H}'$. Formally,

$$t_\ell^{\text{clean}} \notin \mathcal{T}_\ell^{\mathcal{H}'}.$$

Thus the intervention attempts to impose an inadmissible value within the wrong option-space.

18.6 Event-Aware Restoration

We now perform event-aware causal tracing.

Stage 1: Event Restoration. We force the occurrence of the early bind and pop events:

$$\text{do}(e_1^{\text{bind}}), \quad \text{do}(e_2^{\text{pop}}),$$

thereby aligning the kernel history:

$$\mathcal{H}' \;\Rightarrow\; \tilde{\mathcal{H}} \approx \mathcal{H}.$$

Stage 2: Texture Restoration. We then restore texture values:

$$\text{do}(T_\ell := t_\ell^{\text{clean}}).$$

Result. Accuracy recovers to near-clean levels, with significantly reduced variance.

Proposition 5 (Kernel Alignment Sufficiency)**.** *If kernel histories are aligned up to layer ℓ, then restoring texture at layer ℓ is sufficient to recover behavior.*

Proof. Kernel alignment ensures $\mathcal{T}_\ell^{\tilde{\mathcal{H}}} = \mathcal{T}_\ell^{\mathcal{H}}$. The restored texture is therefore admissible and can propagate causally. □

18.7 Partial Recovery Explained

Cases of partial recovery correspond to partial kernel overlap. If only some early events are restored, the option-space intersection

$$\Omega(\mathcal{H}) \cap \Omega(\mathcal{H}')$$

is nonempty but restricted. Textures lying in this intersection yield graded recovery, explaining empirically observed block-structured effects.

18.8 Implications for Circuit Claims

This case study refines circuit-level explanations. Circuits should be understood as event-conditioned mechanisms: a circuit operates only within a specific kernel history.

Claims such as "head h implements IOI" are incomplete unless qualified by the kernel in which h operates.

Remark 8. The correct explanatory unit is not a component in isolation, but a component embedded in a particular event-history.

18.9 Summary

This example demonstrates that:

- Standard causal tracing probes both texture and event channels without distinguishing them.
- Restoration failures diagnose kernel mismatch, not absence of mechanism.
- Event-aware interventions convert partial explanations into robust ones.

The next section extends this analysis temporally, examining how kernel histories emerge and stabilize during training.

19 Training Dynamics as Kernel Formation

Mechanistic interpretability is typically applied to trained models, treating learned structure as static. However, many empirical phenomenamost notably grokking, sudden circuit emergence, and regime switchingare difficult to explain without examining how event-level structure forms during training.

In this section, we reinterpret training as the progressive construction of kernel histories. Learning is not merely the optimization of texture values, but the gradual stabilization of event-nodes that irreversibly constrain future computation.

19.1 Texture Learning vs. Kernel Formation

We distinguish two learning processes:

Texture learning. Continuous adjustment of parameters within an existing computational regime. This corresponds to gradient descent refining weights without altering the qualitative structure of computation.

Kernel formation. Discrete commitment to a new regime, realized as the emergence or stabilization of event-nodes. Kernel formation changes which computations are admissible at all.

This distinction explains why training loss often decreases smoothly while generalization improves discontinuously.

Assumption 2 (Event Stabilization)**.** *Event-nodes are metastable during early training and become irreversible only after sufficient evidence accumulates.*

19.2 Grokking as Event Commitment

Grokking refers to the phenomenon where a model fits training data early but generalizes only after a long delay, often accompanied by a sudden drop in test loss.

Under the present framework, grokking corresponds to delayed kernel commitment.

Early phase. The model relies on texture-level heuristics that interpolate within the training set. Event-nodes corresponding to abstract rules are not yet stabilized.

Transition. A critical training iteration triggers an event-node committing to a new regime (e.g., arithmetic structure, symmetry exploitation).

Late phase. Once committed, texture learning proceeds rapidly within the new kernel, yielding strong generalization.

Theorem 2 (Grokking as Kernel Transition)**.** *Let $\mathcal{H}_t$ denote the kernel history at training step t. Grokking occurs when there exists a step $t^\star$ such that:*

$$\mathcal{H}_{t^\star -} \neq \mathcal{H}_{t^\star +},$$

and $\mathcal{H}_{t^\star +}$ admits a strictly smaller hypothesis space consistent with the task.

Sketch. Prior to $t^\star$, multiple kernels remain compatible with observed data. At $t^\star$, accumulated evidence renders all but one kernel inadmissible, triggering an irreversible commitment. □

19.3 Sudden Circuit Emergence

Mechanistic interpretability studies frequently observe circuits appearing abruptly rather than gradually. From a texture-only perspective, this is surprising.

In the event-historical view, this is expected: circuits correspond to event-conditioned pathways that become active only after kernel formation.

Proposition 6 (Circuit Visibility)**.** *A circuit becomes mechanistically visible if and only if the kernel history that enables it has stabilized.*

Thus, interpretability does not merely reveal structure; it reveals structure that has already become irreversible.

19.4 Training-Time Causal Tracing

Standard causal tracing is applied post hoc. However, event-aware analysis suggests a training-time analogue.

Protocol. At checkpoints $t_1 < t_2 < \cdots < t_k$:

1. Perform event-aware tracing on a fixed input.
2. Record which event-nodes are necessary for correct output.
3. Track stabilization of these events across time.

Prediction. Event-nodes exhibit hysteresis: once stabilized, they resist reversal even under later parameter noise.

Remark 9. This predicts that pruning or ablation early in training may prevent kernel formation, while the same intervention late in training has little effect.

19.5 Algorithmic Mixture During Training

Early in training, models often exhibit algorithmic pluralism: multiple kernels coexist, each weakly supported by data.

We model this as a distribution over potential kernels $\{\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(k)}\}$, with soft selection mediated by event uncertainty.

Kernel selection is itself an event:

$$e_{\text{select}} : \{\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(k)}\} \to \mathcal{H}^{\star}.$$

Once selected, alternative kernels are excluded, explaining the loss of heuristic diversity after grokking.

19.6 Implications for Interpretability Practice

This analysis yields concrete methodological guidance:

- Interpretability applied too early may miss future event-nodes.
- Interpretability applied too late may obscure kernel alternatives.
- Training-aware tracing can identify incipient regimes before commitment.

19.7 Summary

Training is not merely parameter optimization but the construction of irreversible event-histories. Mechanistic structure emerges when kernels stabilize, explaining grokking, circuit suddenness, and regime switching.

The next section generalizes these insights beyond neural networks, connecting event-historical structure to resilience in large-scale knowledge systems.

20 Training Dynamics as Kernel Formation

Mechanistic interpretability is typically applied to trained models, treating learned structure as static. However, many empirical phenomenamost notably grokking, sudden circuit emergence, and regime switchingare difficult to explain without examining how event-level structure forms during training.

In this section, we reinterpret training as the progressive construction of kernel histories. Learning is not merely the optimization of texture values, but the gradual stabilization of event-nodes that irreversibly constrain future computation.

20.1 Texture Learning vs. Kernel Formation

We distinguish two learning processes:

Texture learning. Continuous adjustment of parameters within an existing computational regime. This corresponds to gradient descent refining weights without altering the qualitative structure of computation.

Kernel formation. Discrete commitment to a new regime, realized as the emergence or stabilization of event-nodes. Kernel formation changes which computations are admissible at all.

This distinction explains why training loss often decreases smoothly while generalization improves discontinuously.

Assumption 3 (Event Stabilization)**.** *Event-nodes are metastable during early training and become irreversible only after sufficient evidence accumulates.*

20.2 Grokking as Event Commitment

Grokking refers to the phenomenon where a model fits training data early but generalizes only after a long delay, often accompanied by a sudden drop in test loss.

Under the present framework, grokking corresponds to delayed kernel commitment.

Early phase. The model relies on texture-level heuristics that interpolate within the training set. Event-nodes corresponding to abstract rules are not yet stabilized.

Transition. A critical training iteration triggers an event-node committing to a new regime (e.g., arithmetic structure, symmetry exploitation).

Late phase. Once committed, texture learning proceeds rapidly within the new kernel, yielding strong generalization.

Theorem 3 (Grokking as Kernel Transition)**.** *Let $\mathcal{H}_t$ denote the kernel history at training step t. Grokking occurs when there exists a step $t^\star$ such that:*

$$\mathcal{H}_{t^\star -} \neq \mathcal{H}_{t^\star +},$$

and $\mathcal{H}_{t^\star+}$ admits a strictly smaller hypothesis space consistent with the task.

Sketch. Prior to $t^\star$, multiple kernels remain compatible with observed data. At $t^\star$, accumulated evidence renders all but one kernel inadmissible, triggering an irreversible commitment. □

20.3 Sudden Circuit Emergence

Mechanistic interpretability studies frequently observe circuits appearing abruptly rather than gradually. From a texture-only perspective, this is surprising.

In the event-historical view, this is expected: circuits correspond to event-conditioned pathways that become active only after kernel formation.

Proposition 7 (Circuit Visibility)**.** *A circuit becomes mechanistically visible if and only if the kernel history that enables it has stabilized.*

Thus, interpretability does not merely reveal structure; it reveals structure that has already become irreversible.

20.4 Training-Time Causal Tracing

Standard causal tracing is applied post hoc. However, event-aware analysis suggests a training-time analogue.

Protocol. At checkpoints $t_1 < t_2 < \cdots < t_k$:

1. Perform event-aware tracing on a fixed input.
2. Record which event-nodes are necessary for correct output.
3. Track stabilization of these events across time.

Prediction. Event-nodes exhibit hysteresis: once stabilized, they resist reversal even under later parameter noise.

Remark 10. This predicts that pruning or ablation early in training may prevent kernel formation, while the same intervention late in training has little effect.

20.5 Algorithmic Mixture During Training

Early in training, models often exhibit algorithmic pluralism: multiple kernels coexist, each weakly supported by data.

We model this as a distribution over potential kernels $\{\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(k)}\}$, with soft selection mediated by event uncertainty.

Kernel selection is itself an event:

$$e_{\text{select}} : \{\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(k)}\} \to \mathcal{H}^\star.$$

Once selected, alternative kernels are excluded, explaining the loss of heuristic diversity after grokking.

20.6 Implications for Interpretability Practice

This analysis yields concrete methodological guidance:

- Interpretability applied too early may miss future event-nodes.
- Interpretability applied too late may obscure kernel alternatives.
- Training-aware tracing can identify incipient regimes before commitment.

20.7 Summary

Training is not merely parameter optimization but the construction of irreversible event-histories. Mechanistic structure emerges when kernels stabilize, explaining grokking, circuit suddenness, and regime switching.

The next section generalizes these insights beyond neural networks, connecting event-historical structure to resilience in large-scale knowledge systems.

21 Knowledge Networks as Event-Historical Systems

The causal framework developed thus far is not specific to neural networks. Any large-scale information system that evolves through irreversible decisions admits an event-historical analysis. In this section we extend the theory to knowledge networks, such as encyclopedias, citation graphs, and collaborative repositories, and show that their resilience depends primarily on event-level structure.

21.1 Texture and Event Structure in Knowledge Systems

Consider a knowledge network represented as a directed graph $\mathcal{G} = (V, E)$, where nodes correspond to informational units (articles, documents, entries) and edges encode semantic or referential relationships.

Texture-nodes. The content of articles, the existence of links, and local formatting choices are texture-level structures. They admit substitution, duplication, and repair without altering the global regime of the system.

Event-nodes. Governance decisions, moderation policies, inclusion criteria, and institutional norms are event-level structures. These decisions restrict which future edits, links, or topics are admissible.

Definition 15 (Knowledge-System Event)**.** *An event in a knowledge network is an irreversible decision that restricts the admissible future states of the network, independent of the current content.*

Examples include:

- Banning a class of sources

- Locking an article category
- Enforcing notability thresholds
- Centralizing editorial authority

21.2 Resilience Beyond Redundancy

A common intuition is that knowledge systems are resilient if they contain redundant content. From a texture-only perspective, this is plausible: duplicated information can compensate for local damage.

However, redundancy alone does not guarantee resilience.

Proposition 8 (Event-Dominant Failure)**.** *If an event-level intervention restricts future admissible edits, then texture redundancy cannot restore lost adaptive capacity.*

Sketch. Let Ω_t denote the option-space of admissible future states. Texture redundancy increases the density of states within Ω_t but does not expand Ω_t itself. Event interventions restrict Ω_t, reducing future adaptability regardless of internal redundancy. □

Thus, resilience must be analyzed at the level of option-spaces, not content volume.

21.3 Causal Tracing in Knowledge Networks

The eventtexture distinction enables a form of causal tracing in knowledge systems.

Texture tracing. Removing or restoring articles tests whether specific content mediates knowledge propagation.

Event tracing. Reverting or simulating governance decisions tests whether institutional structures mediate long-term outcomes.

Definition 16 (Event Necessity in Knowledge Networks)**.** *An event e is necessary for outcome Y if removing e while holding content fixed changes the long-term evolution of the network.*

This explains why some systems collapse despite retaining vast amounts of information: the event-history no longer permits repair.

21.4 Irreversibility and Institutional Memory

Event-histories accumulate institutional memory. Even if a policy is formally reversed, downstream effects may persist due to path dependence.

Lemma 3 (Institutional Hysteresis)**.** *Let e be an event restricting option-space at time t. Even if an inverse event e^{-1} occurs at time $t' > t$, the option-space $\Omega_{t'}$ may satisfy:*

$$\Omega_{t'} \subsetneq \Omega_{t^-},$$

due to lost contributors, abandoned topics, or frozen structures.

This mirrors neural training: once kernels stabilize, earlier regimes may become unreachable.

21.5 Algorithmic Mixture in Knowledge Systems

Just as neural networks may implement multiple algorithms, knowledge systems may support multiple epistemic regimes simultaneously.

Examples include:

- Expert-driven vs. crowd-driven editing
- Citation-based vs. narrative-based validation
- Centralized vs. federated governance

These correspond to parallel kernels $\{\mathcal{H}^{(1)}, \ldots, \mathcal{H}^{(k)}\}$ governing interpretation and growth.

Selection among them is itself an event:

$$e_{\text{select}} : \{\mathcal{H}^{(i)}\} \to \mathcal{H}^{\star}.$$

Once selected, alternative epistemic styles may become inadmissible, even if they remain technically possible.

21.6 Design Implications

This analysis yields concrete design principles:

- Preserve event-level reversibility where possible
- Avoid premature collapse of governance diversity
- Treat policy changes as high-impact interventions requiring causal testing

21.7 Summary

Knowledge networks are event-historical systems whose long-term resilience depends more on institutional commitments than on content volume. The same causal tools used to interpret neural networks apply, mutatis mutandis, to social and informational infrastructures.

The final section synthesizes these results, returning to the question of what it means to understand a system mechanistically.

22 Conclusion

Mechanistic interpretability has undergone a decisive shift from descriptive inspection toward causal explanation. Through interventions, counterfactuals, and causal abstractions, it now aspires to recover the algorithms implemented by complex learned systems. However, as we have argued, this causal turn remains conceptually incomplete so long as it treats internal values as the sole bearers

of mechanism. Many of the most salient empirical phenomena in interpretabilitysubstitution failure, block-level mediation, sudden circuit emergence, grokking, and narrow regime validityare difficult or impossible to formalize within a purely state-based ontology.

This work introduced a minimal but principled extension to the causal framework: the explicit distinction between *texture-nodes*, which carry instantaneous values, and *event-nodes*, which encode irreversible commitments that constrain admissible futures. This distinction preserves the empirical strengths of existing methods while resolving their theoretical tensions. Texture-level interventions explain graded, reversible effects; event-level interventions explain regime change, irreversibility, and structural failure.

Within this typed framework, causal mediation decomposes naturally into two channels. Texture-mediated effects describe value propagation within a fixed regime, while event-mediated effects describe kernel selection and option-space restriction. The resulting two-channel mediation theorem clarifies when activation patching and causal tracing are sufficient, when they fail, and why restoration order matters. Substitution failure is no longer mysterious: it arises when values are transplanted into incompatible event-histories.

By interpreting event-nodes through Spherepop operators*Pop*, *Bind*, *Refuse*, and *Collapse*we provided a concrete semantics for regime change that applies uniformly across neural computation, training dynamics, and social knowledge systems. Neural networks do not merely transform representations; they construct irreversible computational histories. Training, in turn, is not merely optimization over parameters, but the gradual stabilization of event-level structure, explaining grokking, sudden circuit emergence, and algorithmic mixture as kernel-level phenomena.

Extending the analysis beyond neural models revealed that the same principles govern large-scale knowledge networks. Content redundancy alone does not ensure resilience; institutional and governance events determine whether systems retain the capacity to adapt, repair, and grow. Knowledge systems, like neural networks, fail catastrophically when event-level commitments prematurely collapse their option-spaces.

Taken together, these results motivate a reconception of mechanistic understanding itself. To understand a system mechanistically is not merely to locate variables whose manipulation changes outputs. It is to reconstruct the irreversible sequence of commitments that shaped the systems present behavior and delimit its future possibilities. Mechanistic interpretability thus becomes an event-historical science: the reverse engineering of constraint, not merely of computation.

This perspective does not diminish the value of existing tools; rather, it situates them within a broader causal ontology that accounts for time, irreversibility, and regime structure. By making events explicit, we gain a framework capable of unifying interpretability, training dynamics, and institutional analysis under a single causal language.

In this sense, mechanistic interpretability is itself a Spherepop process. Each successful intervention permanently constrains the space of viable explanations. Understanding accumulates not as a static map of components, but as an irreversible history of ruled-out possibilities. What remains is not merely what the system does, but what it can no longer doand it is in these absences, exclusions, and commitments that mechanism ultimately resides.

A Formal Preliminaries

This appendix formalizes the mathematical objects used throughout the paper. All results in subsequent appendices assume these definitions.

A.1 Structural Causal Models

Definition 17 (Structural Causal Model)**.** *A structural causal model (SCM) is a tuple*

$$\mathcal{M} = (V, U, F, P(U)),$$

where:

- $V = \{V_1, \ldots, V_n\}$ *are endogenous variables,*
- $U = \{U_1, \ldots, U_m\}$ *are exogenous variables,*
- $F = \{f_i\}_{i=1}^{n}$ *are structural equations*

$$V_i := f_i(\mathrm{PA}_i, U_i),$$

- $P(U)$ *is a probability distribution over exogenous variables.*

Assumption 4 (Causal Markov Condition)**.** *Each variable V_i is conditionally independent of its non-descendants given its parents* PA_i*.*

A.2 Typed Causal Graphs

Definition 18 (Typed Causal Graph)**.** *A typed causal graph is a tuple*

$$\mathcal{G} = (V, E, \tau),$$

where (V, E) is a directed acyclic graph and

$$\tau : V \to \{\mathbf{T}, \mathbf{E}\}$$

assigns each node a type: texture or event.

Interpretation. Texture-nodes admit value substitution under intervention. Event-nodes rewrite the admissible future structure of the graph itself.

A.3 Option-Spaces and Kernel Histories

Definition 19 (Option-Space)**.** *An option-space Ω_t is the set of admissible future trajectories from time t, partially ordered by refinement.*

Definition 20 (Kernel History)**.** *A kernel history up to time t is a sequence*

$$\mathcal{H}_t = (e_1, e_2, \ldots, e_t),$$

where each e_i is an event-node realization. The kernel history induces an option-space $\Omega_t = \Omega(\mathcal{H}_t)$.

Assumption 5 (Monotonicity)**.** *Kernel histories are monotone:*

$$\Omega_{t+1} \subseteq \Omega_t.$$

B Proofs of Main Results

B.1 Proof of the Two-Channel Mediation Theorem

Theorem 4 (Two-Channel Mediation)**.** *Let X be an input variable, T a texture variable, E an event variable, and Y an output. Then the total causal effect decomposes as*

$$\mathrm{TE}(X \to Y) = \mathrm{ME}^T(X \to Y) + \mathrm{ME}^E(X \to Y),$$

where texture mediation suffices iff $P(E \mid \mathrm{do}(X)) = P(E)$.

Proof. The total effect is defined as:

$$\mathrm{TE}(X \to Y) = \mathbb{E}[Y \mid \mathrm{do}(X = x_1)] - \mathbb{E}[Y \mid \mathrm{do}(X = x_0)].$$

Introduce the mediators T and E:

$$\mathbb{E}[Y \mid \mathrm{do}(X = x)] = \sum_{t,e} P(t, e \mid \mathrm{do}(X = x))\, \mathbb{E}[Y \mid \mathrm{do}(T = t, E = e)].$$

Subtracting the two expressions yields:

$$\mathrm{TE} = \sum_{t,e} \Big(P(t, e \mid \mathrm{do}(X = x_1)) - P(t, e \mid \mathrm{do}(X = x_0)) \Big) \mathbb{E}[Y \mid \mathrm{do}(T = t, E = e)].$$

Decompose the joint difference:

$$P(t, e \mid \mathrm{do}(X)) = P(t \mid e, \mathrm{do}(X)) P(e \mid \mathrm{do}(X)).$$

Define:

- ME^E: variation due to $P(e \mid \mathrm{do}(X))$,
- ME^T: variation due to $P(t \mid e, \mathrm{do}(X))$ holding $P(e)$ fixed.

If $P(E \mid \mathrm{do}(X)) = P(E)$, the event-mediated term vanishes, and the effect is fully texture-mediated. Conversely, if E varies under $\mathrm{do}(X)$, texture mediation alone is insufficient. □

B.2 Event Irreversibility

Theorem 5 (Event Irreversibility)**.** *Let e be an event-node occurring at time t. Then for all texture interventions at times $t' > t$, the induced option-space satisfies:*

$$\Omega_{t'} \subseteq \Omega_{t^-}.$$

Proof. By definition, an event-node rewrites the admissible graph structure, removing at least one future trajectory from Ω_{t^-}. Texture interventions operate within the existing graph and cannot reintroduce deleted trajectories. Hence the inclusion is strict. □

B.3 Substitution Failure

Proposition 9 (Substitution Failure)**.** *Let $t^\star$ be a texture value admissible under kernel history $\mathcal{H}$ but not under $\mathcal{H}'$. Then the intervention* $\mathrm{do}(T := t^\star)$ *under $\mathcal{H}'$ is ill-defined or yields incoherent behavior.*

Proof. Admissibility of $t^\star$ requires $t^\star \in \Omega(\mathcal{H})$. Since $\Omega(\mathcal{H}') \subsetneq \Omega(\mathcal{H})$, $t^\star$ may violate constraints encoded by $\mathcal{H}'$. The intervention either cannot be realized or produces behavior outside the modeled regime. □

C Derivations and Dynamical Results

C.1 Training as Kernel Formation

Let θ_t denote parameters at training step t. Standard optimization updates $\theta_t \mapsto \theta_{t+1}$ continuously. However, kernel history $\mathcal{H}_t$ changes only when an event-node stabilizes.

Definition 21 (Kernel Stabilization)**.** *An event e stabilizes at time t if for all $t' > t$,*

$$P(e \text{ occurs at } t') \approx 1.$$

C.2 Grokking as Phase Transition

Define the hypothesis space $\mathcal{H}_t$ compatible with kernel history $\mathcal{H}_t$.

Proposition 10 (Hypothesis Collapse)**.** *If an event e excludes all but one hypothesis class compatible with the task, then generalization error decreases discontinuously.*

Proof. Before stabilization, $\mathcal{H}_t$ contains both memorizing and rule-based hypotheses. Stabilization of e eliminates the former, causing an abrupt reduction in test error. □

C.3 Collapse as Quotienting

Let X be a representation space and $q : X \to X/\!\sim$ a quotient map.

Definition 22 (Collapse Operator)**.** *A collapse event identifies equivalence classes under $\sim$, reducing the effective dimensionality of X.*

Proposition 11 (Action Reduction Under Collapse)**.** *Let $S[\mathcal{H}]$ denote accumulated action. Then a collapse event q satisfies:*

$$S[\mathcal{H} \circ q] = S[\mathcal{H}] - \log |\ker(q)|.$$

Proof. Collapse removes distinctions that no longer affect future action, reducing informational degrees of freedom. The logarithmic term accounts for identified equivalence classes. □

C.4 Why Steering Is Brittle

Let $M \subset \mathbb{R}^d$ be the semantic manifold of admissible representations.

Theorem 6 (Tangent Constraint)**.** *A steering vector v preserves semantic coherence iff $v \in T_x M$.*

Proof. Components orthogonal to $T_x M$ correspond to directions violating kernel constraints, forcing later layers either to project back onto M or to follow incoherent trajectories. □

D Appendix D: Worked Mechanistic Case Study

D.1 Task Description

We consider the Indirect Object Identification (IOI) task, exemplified by sentences of the form:

> *"John gave Mary the book."*

The model must identify *Mary* as the indirect object. This task is well studied in mechanistic interpretability due to its clear syntactic structure and sensitivity to internal mechanisms.

D.2 Model and Notation

We analyze a transformer model with L layers. Let:

- X_0 denote the input token embeddings,
- X_ℓ the residual stream at layer ℓ,
- $H_{\ell,i}$ the i-th attention head at layer ℓ,
- M_ℓ the MLP at layer ℓ.

We distinguish:

- Texture variables: activations in X_ℓ, outputs of heads and MLPs,
- Event variables: discrete commitments that constrain which relations are considered admissible (e.g. binding verbargument structure).

D.3 Kernel History for IOI

Empirical circuit analyses suggest the following kernel history:

Layer 1 (Event Formation).

- e_1^{bind}: Bind the verb token *gave* to candidate noun positions.
- e_2^{pop}: Exclude subject position from indirect object candidates.

These events restrict the option-space to trajectories where only post-verbal noun phrases are considered.

Layer 2 (Aggregation and Refinement).

- e_3^{collapse}: Collapse multiple candidate positions into a single salience-weighted representation.
- e_4^{meld}: Combine syntactic and semantic cues to select the indirect object.

Thus the kernel history is:

$$\mathcal{H} = (e_1^{\text{bind}}, e_2^{\text{pop}}, e_3^{\text{collapse}}, e_4^{\text{meld}}).$$

D.4 Standard Texture-Level Ablation

We first perform a standard ablation:

$$\text{do}(H_{1,0} := 0),$$

where $H_{1,0}$ is an attention head previously identified as important for IOI.

Observation. Accuracy drops from approximately 45% to 12%.

Texture Interpretation. This suggests that $H_{1,0}$ carries information relevant to IOI.

Event Interpretation. $H_{1,0}$ implements the event e_1^{bind}. Its ablation does not merely degrade a value; it prevents the verbargument binding regime from forming.

D.5 Naive Texture Restoration

We next attempt texture-level restoration after corrupting the input:

$$\text{do}(X_1 := X_1^{\text{clean}}).$$

Observation. Accuracy recovers only partially (to $\sim 18\%$).

Explanation. The restored texture is incompatible with the corrupted kernel history $\mathcal{H}'$, in which e_1^{bind} never occurred. The restored values lie outside the admissible option-space $\Omega(\mathcal{H}')$.

This is a concrete instance of substitution failure.

D.6 Event-Aware Restoration

We now perform a two-stage intervention:

$$\text{do}(e_1^{\text{bind}}) \;\text{ then }\; \text{do}(X_1 := X_1^{\text{clean}}).$$

Observation. Accuracy recovers to $\sim 42\%$, near the clean baseline.

Explanation. Restoring the event reconstitutes the correct kernel history, expanding the option-space so that the restored texture becomes admissible.

Proposition 12 (Event Precedence)**.** *In IOI, restoring event-level structure is a necessary precondition for successful texture restoration.*

D.7 Why Some Heads Are Catastrophic

Ablating certain heads causes near-total failure, while others have minimal impact.

Event-level heads. Heads implementing e_1^{bind} or e_2^{pop} determine the regime. Their removal collapses the option-space.

Texture-level heads. Heads refining salience or probability distributions operate within the established kernel. Their ablation degrades performance smoothly.

This distinction cannot be captured by importance scores alone.

D.8 Algorithmic Mixture Interpretation

Early layers may support multiple candidate kernels:

- $\mathcal{H}^{(1)}$: rule-based syntactic parsing,
- $\mathcal{H}^{(2)}$: surface heuristic (e.g. proximity-based guessing).

Early in training, both kernels coexist. Event e_{select} commits to $\mathcal{H}^{(1)}$, explaining sudden generalization improvements and circuit stabilization.

D.9 Summary

This case study demonstrates that:

- Substitution failure arises from kernel incompatibility, not noise,
- Causal responsibility separates cleanly into event and texture roles,
- Mechanistic circuits correspond to stabilized kernel histories, not merely activation patterns.

The eventtexture framework provides strictly more explanatory power than state-based analysis while remaining empirically testable.

E Appendix E: Automated Detection of Event-Nodes

This appendix develops systematic procedures for identifying event-level structure in neural networks. The goal is to demonstrate that the eventtexture distinction is not purely post hoc, but admits algorithmic detection criteria grounded in causal behavior.

E.1 Problem Statement

Manual identification of event-nodes does not scale to large models. Standard importance metrics (e.g. gradients, attribution scores) fail to distinguish regime-selecting components from value-refining components.

We therefore seek *causal signatures* that distinguish event-nodes from texture-nodes.

E.2 Restoration Order Sensitivity

The first diagnostic exploits asymmetry in restoration order.

Definition 23 (Restoration Order Sensitivity)**.** *Let A and B be internal components. Define:*

$$R(A \to B) := \textit{recovery under } \mathrm{do}(A) \textit{ then } \mathrm{do}(B),$$

and analogously $R(B \to A)$. Component A is event-level relative to B if:

$$R(A \to B) \gg R(B \to A).$$

Interpretation. Event-level structure must be restored before texture-level refinement can succeed. Texture restoration without regime restoration fails.

Proposition 13 (Event Precedence Criterion)**.** *If restoration order matters, the earlier-restored component is event-level with respect to the later one.*

E.3 Ablation Asymmetry

A second diagnostic exploits asymmetry between ablation and restoration.

Definition 24 (Ablation Asymmetry)**.** *Component C exhibits ablation asymmetry if:*

- *Ablation causes catastrophic failure, but*
- *Partial restoration yields minimal recovery unless additional components are restored.*

Explanation. Catastrophic ablation indicates collapse of the option-space, not mere value degradation. Partial restoration fails because the kernel history remains incompatible.

Lemma 4. *Pure texture-nodes exhibit approximately symmetric ablation and restoration effects.*

E.4 Training-Time Stability Analysis

Event-nodes exhibit hysteresis across training time.

Definition 25 (Event Stability)**.** *Let C be a component. Define stability as:*

$$S(C,t) = \mathbb{E}[\textit{performance} \mid \mathrm{do}(C := 0) \textit{ at time } t].$$

C is event-level if $S(C,t)$ decreases sharply after a critical training step and does not recover.

Prediction. Event-nodes:

- Are fragile early in training,
- Become increasingly irreversible,
- Resist later parameter noise.

E.5 Kernel Inference Algorithm

We summarize an automated kernel inference procedure.

Input. Model $\mathcal{N}$, task distribution $\mathcal{D}$, intervention set $\mathcal{I}$.

Procedure.

1. Perform texture-level causal tracing to identify candidate components.
2. For each candidate:
    - Measure restoration order sensitivity,
    - Measure ablation asymmetry,
    - Measure training-time stability.
3. Classify components as event- or texture-level via thresholding.

Output. A partial ordering of components by event precedence.

E.6 Limits and Failure Modes

Not all components admit clean classification. Hybrid nodes may exist that partially affect regime selection and partially refine values. Such nodes appear empirically rare but theoretically possible.

E.7 Summary

This appendix demonstrates that event-nodes can be detected via observable causal asymmetries. The framework therefore supports scalable mechanistic analysis rather than purely interpretive labeling.

F Appendix F: Comparison to Existing Interpretability Frameworks

This appendix situates the eventhistorical framework relative to existing approaches in mechanistic interpretability and adjacent fields. The intent is not to displace these methods, but to clarify their scope, limitations, and points of integration.

F.1 Circuits-Based Interpretability

The circuits framework identifies collections of neurons, attention heads, and MLPs whose interactions implement specific behaviors.

Common ground. Both frameworks:

- Emphasize localized internal mechanisms,
- Rely on causal intervention rather than correlation,
- Aim to reverse engineer implemented algorithms.

Key difference. Circuits are fundamentally *static*. They describe which components interact, but not how regime selection or irreversibility emerges over time.

Proposition 14. *A circuit description corresponds to a snapshot of a stabilized kernel history.*

Implication. Event-nodes explain why some circuits appear suddenly during training and why ablating certain components causes catastrophic failure rather than gradual degradation.

F.2 Sparse Autoencoders and Feature Bases

Sparse autoencoders (SAEs) seek interpretable basis directions in activation space.

Strength. SAEs excel at discovering *texture-level* structure: reusable, localized semantic features.

Limitation. They do not identify which features select regimes versus refine values within a regime.

Remark 11. Event-level structure may not correspond to sparse directions at all, but to discrete gating, masking, or binding operations.

Thus, SAEs and event-historical analysis are complementary: SAEs decompose texture space, while event analysis decomposes kernel space.

F.3 Knowledge Editing and ROME-Style Interventions

Knowledge editing methods (e.g. ROME) modify specific internal weights to change factual outputs.

Observation. Such edits often have global, persistent effects, altering multiple downstream behaviors.

Interpretation. These interventions frequently operate at the event-level: they change which factual regime is selected, not merely the strength of a belief.

Proposition 15. *Successful knowledge edits correspond to kernel modifications rather than texture substitutions.*

This explains why some edits generalize broadly while others degrade unrelated capabilities.

F.4 Representation Engineering and Activation Steering

Representation engineering attempts to control models by manipulating internal representations.

Empirical fact. Prompting and fine-tuning outperform steering for robust control.

Explanation. Steering perturbs texture values within a fixed kernel. Prompting induces early event-level commitments, selecting different regimes.

Remark 12. This clarifies why steering is brittle: it operates downstream of kernel selection.

F.5 Functionalism and State Equivalence

Functionalist accounts identify systems by inputoutput behavior or current state.

Limitation. State equivalence collapses histories that differ in irreversible commitments.

Event-historical correction. Two systems may be functionally identical at a moment while diverging under intervention due to different kernel histories.

F.6 Summary

Existing frameworks each capture important aspects of neural mechanism, but none explicitly model irreversible regime selection. The eventtexture framework integrates these approaches by:

- Treating circuits as stabilized kernels,
- Treating features as texture structure,
- Treating edits and prompts as event interventions.

This unification preserves empirical success while extending explanatory reach.

G Appendix G: Spherepop Axiomatization of Event-Historical Causality

This appendix formalizes the relationship between the eventtexture framework developed in the main text and the Spherepop / RSVP axiomatic system. The purpose is not to introduce new machinery, but to show that the causal structures analyzed throughout the paper are instances of a more general event-historical calculus.

G.1 Option-Spaces as Primitive Objects

We take the option-space to be the fundamental ontological unit.

Definition 26 (Option-Space)**.** *An option-space Ω is a partially ordered set of admissible future trajectories, ordered by refinement. Elements of Ω represent equivalence classes of future system evolutions.*

The evolution of a system corresponds to a monotone sequence:

$$\Omega_0 \supseteq \Omega_1 \supseteq \cdots \supseteq \Omega_t.$$

Assumption 6 (Irreversibility)**.** *All admissible dynamics satisfy:*

$$\Omega_{t+1} \subseteq \Omega_t.$$

This assumption is not thermodynamic in origin; it is structural. It states that commitments restrict futures.

G.2 Spherepop Event Operators

Spherepop defines four primitive operators on option-spaces. We restate them here in causal terms.

G.2.1 Pop (Exclusion)

Definition 27 (Pop)**.** *A pop event e_{pop} is an operator:*

$$e_{pop} : \Omega \to \Omega' \subsetneq \Omega,$$

which permanently excludes at least one admissible future.

Causal interpretation. Pop corresponds to categorical pathway exclusion: masking, pruning, gating, or hard constraints. In causal graphs, pop rewrites the graph by deleting edges or nodes.

G.2.2 Bind (Dependency Formation)

Definition 28 (Bind)**.** *A bind event e_{bind} introduces a dependency relation $\prec$ over futures in Ω, restricting admissible orderings.*

Causal interpretation. Bind corresponds to establishing precedence or structural coupling, such as verbargument binding or residual stream dependency.

G.2.3 Refuse (Principled Non-Action)

Definition 29 (Refuse)**.** *A refuse event e_{refuse} excludes futures not by feasibility but by constraint:*

$$\Omega' = \{\omega \in \Omega : \omega \textit{ violates rule } R\}.$$

Causal interpretation. Refuse corresponds to safety filters, norm enforcement, or learned inhibitory structure. Its defining feature is that the excluded futures remain technically reachable but are ruled out.

G.2.4 Collapse (Quotienting)

Definition 30 (Collapse)**.** *A collapse event $e_{collapse}$ identifies an equivalence relation $\sim$ on Ω and replaces Ω with the quotient $\Omega/\sim$.*

Causal interpretation. Collapse corresponds to abstraction, pooling, compression, and dimensionality reduction. Distinctions that no longer affect future action are erased.

G.3 Kernel Histories as Spherepop Programs

A Spherepop program is a finite sequence of event operators.

Definition 31 (Kernel History)**.** *A kernel history is a sequence:*

$$\mathcal{H} = (e_1, e_2, \ldots, e_k),$$

where each e_i is one of the four Spherepop operators.

The induced option-space is:

$$\Omega(\mathcal{H}) = e_k \circ \cdots \circ e_1(\Omega_0).$$

Proposition 16. *Every kernel history induces a unique admissibility structure on texture variables.*

Proof. Each event operator restricts or quotients the option-space, thereby restricting which texture assignments remain consistent. Composition preserves uniqueness. □

G.4 Mapping to Typed Causal Graphs

We now formalize the correspondence used implicitly throughout the paper.

Theorem 7 (SpherepopCausal Graph Correspondence)**.** *Every typed causal graph $(\mathcal{G}, \tau)$ induces a Spherepop program, and every Spherepop program induces an equivalence class of typed causal graphs under graph isomorphism.*

Sketch. Event-nodes correspond to Spherepop operators acting on option-spaces. Texture-nodes correspond to coordinates within the resulting quotient space. Graph rewrites implement pop and bind; quotienting implements collapse; constraint edges implement refuse. □

G.5 Mechanistic Interpretability as Spherepop

Finally, we formalize the meta-claim of the paper.

Proposition 17 (Interpretability as Event-History Construction)**.** *Mechanistic interpretability is the construction of a kernel history $\mathcal{H}^*$ such that:*

1. *$\Omega(\mathcal{H}^*)$ excludes all hypotheses inconsistent with observed interventions,*
2. *For all tested interventions, $\mathcal{H}^*$ predicts outcomes correctly.*

Interpretation. Each successful experiment permanently excludes explanations. Failed hypotheses are popped; spurious distinctions are collapsed; necessary dependencies are bound.

G.6 Consequences

This axiomatization yields several immediate consequences:

- Mechanistic understanding is inherently irreversible.
- Explanation quality corresponds to option-space reduction.
- Two models are equivalent iff they induce the same kernel history up to admissible reparameterization.

G.7 Summary

Spherepop provides a minimal, formal language for expressing the event-historical commitments implicit in causal mechanistic interpretability. Rather than adding metaphysics, it clarifies the structure already required to explain irreversibility, regime change, and substitution failure.

In this sense, Spherepop is not an alternative to causal explanation, but its completion.